Urban Traffic Controller

Large-scale optimisation of traffic-signal timings
with a parallel genetic algorithm on Cyfronet Ares

Grzegorz Mróz Katarzyna Kucharczyk

June 2026

Contents

Abstract

We present an end-to-end system that designs road networks, optimises their traffic-signal timings, and scales the optimisation across a supercomputer. A microscopic traffic simulator (SUMO) scores a candidate signal plan by the *total vehicle delay* it produces; a genetic algorithm (GA) searches the space of green-phase durations to minimise that delay. Because every fitness evaluation is an independent SUMO run, the population is evaluated in parallel with MPI, turning a sequential search into a high-performance computing (HPC) workload. On a single 8×8 city the optimised plan cuts total vehicle delay by 27.3 % versus fixed-time

signals. We empirically justify the choice of a GA by comparing it, on an equal evaluation budget, against random search, hill climbing and simulated annealing — the GA reaches the deepest improvement. On Cyfronet Ares the parallel evaluator delivers a 26.96× **speedup on 47 workers** and sustains up to 45.8 evaluations/s, while a problem-size study shows the cost per evaluation growing gracefully from a 9-intersection grid to a 1024-intersection city. An interactive dashboard ties the pieces together: build a network, optimise it, and watch the before/after traffic.

1 Problem statement

Traffic lights at an intersection cycle through a fixed sequence of phases. The duration of each *green* phase decides how much of the cycle a given movement receives. Poorly chosen durations force vehicles to wait at red while a conflicting approach sits empty, wasting time and fuel across the whole network. The classical engineering answer — fixed-time plans from rules of thumb — ignores the actual demand pattern of a specific city.

We treat signal timing as a black-box optimisation problem. Let a city contain intersections $i = 1, \ldots, N$, each with g_i controllable green phases. A *plan* is the vector of all green durations,

$$\mathbf{x} = (x_1, \ldots, x_n), \qquad n = \sum_{i=1}^{N} g_i, \qquad x_j \in [\, g_{\min}, g_{\max} \,] \subset \mathbb{Z}.$$

Yellow and all-red phases stay fixed, so every plan we generate is a valid, safe signal program. The quality of a plan is the *total vehicle delay* it produces over a simulated horizon T:

$$D(\mathbf{x}) \;=\; \sum_{v \in V} \int_0^T \mathbf{1}[\,\text{vehicle } v \text{ is halting at time } t\,]\; dt, \tag{1}$$

i.e. the cumulative vehicle-seconds spent stopped. We seek the plan $\mathbf{x}^\star = \arg\min_{\mathbf{x}} D(\mathbf{x})$. The objective has no closed form: $D(\mathbf{x})$ is only available by running a full microscopic traffic simulation, which makes the problem a natural fit for population-based, derivative-free search — and an expensive one, motivating parallelism.

2 Tools and methods

SUMO (Simulation of Urban MObility) — an open-source microscopic traffic simulator. Each vehicle is modelled individually; the simulator reports per-step halting counts from which Eq. (1) is computed. Networks are generated with `netgenerate` and demand with `randomTrips.py`. On the cluster SUMO is driven in-process through the `libsumo` bindings, avoiding socket overhead.

DEAP — a Python evolutionary-computation framework providing the GA operators (selection, crossover, mutation) we configure for the integer genome.

mpi4py — MPI bindings for Python. We use `MPIPoolExecutor` from `mpi4py.futures` to evaluate the population in a master–worker pattern.

Dash / Plotly — the interactive dashboard front-end: build a network, launch the optimiser, and visualise the before/after traffic.

Cyfronet Ares — the HPC platform (partition `plgrid`). SUMO v1.20.0 is built from source with `libsumo`; OpenMPI 4.1.6 launches the ranks via PMIx.

3 System architecture

The system is a single pipeline that can run serially on a laptop or in parallel on the cluster without code changes:

$$\boxed{\text{Dashboard}} \rightarrow \boxed{\text{Genetic algorithm}} \rightarrow \boxed{\text{SUMO simulation}}$$

The dashboard builds or uploads a network; the GA proposes signal plans; SUMO scores each plan. Crucially, the GA dispatches every evaluation through an abstract `map_fn`. Locally this is Python's built-in `map` (serial); on the cluster it is `MPIPoolExecutor.map` (parallel). The *same* optimiser code therefore drives both the single-machine demo and the HPC scaling study — only the map function changes.

4 Problem formulation

Genome. A candidate plan is encoded as an integer vector: one gene per controllable green phase, holding its duration in seconds and bounded by $[g_{\min}, g_{\max}] = [5\,\text{s}, 60\,\text{s}]$. For an 8×8 grid this yields $n = 128$ genes; for a 32×32 city, $n = 2048$. Because only green durations are encoded and the protective yellow/red phases are untouched, the decoder always emits a legal SUMO signal program.

Fitness. Decoding a genome writes the durations into the network's signal programs; one headless SUMO run then returns the total vehicle delay of Eq. (1). Lower is better. A single *baseline* run with the default fixed-time plan provides the reference against which every result is reported as an improvement percentage.

5 The optimisation algorithm

5.1 Why a genetic algorithm

The objective is non-differentiable, noisy and multimodal: small changes in one intersection's timing interact non-linearly with traffic arriving from its neighbours. There are no usable gradients. A genetic algorithm is well suited because it (i) needs only the scalar fitness, (ii) maintains a *population* of plans, exploring many regions of the search space at once, and (iii) exposes exactly the parallelism we need — the whole population is evaluated independently each generation. Section 6 backs this choice with an empirical comparison against simpler search strategies.

5.2 Operators

Selection tournament selection (size 3): pick the fitter of randomly drawn plans, biasing reproduction towards good solutions while keeping diversity.

Crossover uniform crossover: each child gene is taken from either parent with equal probability, recombining good per-intersection timings.

Mutation per-gene uniform reset within $[g_{\min}, g_{\max}]$, which injects fresh durations and prevents premature convergence.

Elitism a Hall of Fame of size 1 carries the best plan unchanged into the next generation, so the optimum never regresses — the result can never be worse than the baseline.

5.3 Pseudocode

```
initialise population P of random integer plans
evaluate D(x) for every x in P        # via map_fn -> SUMO (parallel)
record best-so-far in Hall of Fame
for generation = 1 .. G:
    offspring <- tournament_select(P)
    offspring <- uniform_crossover(offspring, p_cx)
    offspring <- uniform_reset_mutation(offspring, p_mut)
    evaluate D(x) for every new x       # via map_fn -> SUMO (parallel)
    P <- offspring + elite(Hall of Fame)
    update Hall of Fame
return best plan in Hall of Fame
```

The only line that touches the cluster is `evaluate` — it submits the generation's plans through `map_fn`. Everything else (selection, recombination, bookkeeping) is cheap and runs on the master.

6 Algorithm comparison: why the GA earns its place

A reviewer may reasonably ask whether the GA's machinery is worth it, or whether a simpler search would do as well. To answer this empirically we ran four algorithms on the *same* objective (the 8×8 city, baseline delay 39 473 veh s) and gave each the *same evaluation budget* — the GA's own evaluation count ($\approx$ 1990 SUMO runs). All metaheuristics are seeded with the baseline plan (mirroring the GA's elitism), so none can report a result worse than fixed-time; the question is purely *how far below the baseline each can reach on an equal budget.*

Random search samples random plans and keeps the best.

Hill climbing parallel random-restart local search: each chain accepts a one-gene neighbour only if it improves.

Simulated annealing like hill climbing, but accepts worsening moves with probability $e^{-\Delta/T}$ under a geometric cooling schedule.

Genetic algorithm the production optimiser described above.

Table 1: Final result of each algorithm on an equal evaluation budget (8×8 city, baseline $D = 39\,473$ veh s). Lower delay is better.

Algorithm	Best delay [veh·s]	Improvement [%]	Evaluations
Genetic algorithm	**28689**	**27.3**	1991
Hill climbing	31 693	19.7	1978
Random search	36 010	8.8	1991
Simulated annealing	36 405	7.8	1978
Baseline (fixed-time)	39 473	0.0	1

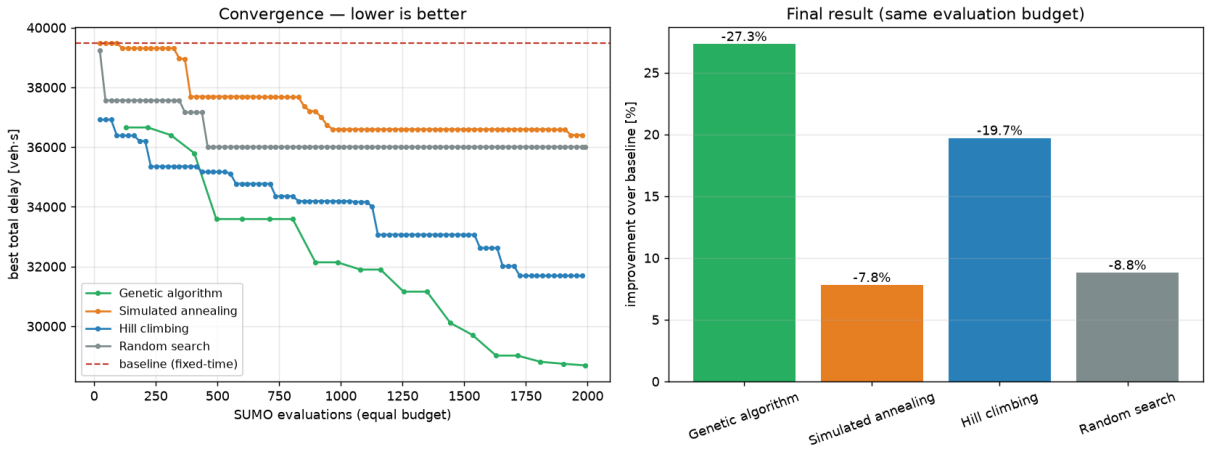

Figure 1: Algorithm comparison on equal budget. *Left:* convergence — best total delay against the number of SUMO evaluations; the GA descends below every competitor. *Right:* final improvement over the fixed-time baseline. The GA reaches $-27.3\,\%$, hill climbing $-19.7\,\%$, while random search and simulated annealing stall near $-8\,\%$.

Reading the result. Table 1 and Fig. 1 make the case quantitatively. The GA improves on the baseline by $27.3\,\%$, clearly the best of the four. Pure random search barely moves the needle ($-8.8\,\%$): the search space is far too large to sample blindly. Hill climbing does much better ($-19.7\,\%$) by exploiting local structure, but its greedy chains get stuck in local optima. Simulated annealing, on this budget and neighbourhood, spends too many of its evaluations accepting worsening moves and never catches up. The GA combines the strengths the others lack — a population that explores broadly, crossover that recombines good per-intersection timings, and elitism that locks in progress — which is exactly why it reaches the deepest improvement. The comparison thus turns "we chose a GA" from an assertion into a measured conclusion.

7 Parallelisation

Each generation requires one SUMO run per plan, and those runs are completely independent. This is the parallelism the system exploits. With a master–worker pattern, the master (MPI rank 0) holds the GA state and scatters the generation's plans to w worker ranks; each worker runs SUMO on its plans and returns the delays; the master gathers them and proceeds.

The achievable speedup is bounded by Amdahl's law. If a fraction f of the work is inherently serial (here: GA bookkeeping, scatter/gather, and the per-batch load imbalance), the speedup on w workers cannot exceed

$$S(w) \;=\; \frac{1}{f + \dfrac{1-f}{w}}. \tag{2}$$

Because evaluation dominates runtime, f is small and the system stays close to linear for moderate worker counts, with efficiency tapering as w grows and the fixed serial cost becomes proportionally larger (Sec. 9.2).

8 Experimental setup

All HPC measurements were taken on **Cyfronet Ares** (PLGrid grant `plglscclass26-cpu`, partition `plgrid`), on a single compute node with up to 48 MPI ranks. SUMO v1.20.0 was *compiled from source* on the cluster with the `libsumo` in-process bindings, because the pre-built `eclipse-sumo` wheel segfaults on Ares compute nodes and ships only the slower socket-based

`traci`. OpenMPI 4.1.6 (matching the GCC 13.2.0 used to build SUMO) launches the ranks, with `SLURM_MPI_TYPE=pmix` so that `srun` starts them via PMIx.

How a study is run. Each study is a self-contained SLURM batch script in `scripts/slurm/`, submitted with `sbatch`. The script loads the cluster modules, activates the `traffic` Conda environment, exports the source-built `SUMO_HOME`, and launches the parallel run:

```
module load miniconda3/24.5.0-0
module load openmpi/4.1.6-gcc-13.2.0   # libmpi + PMIx for srun-launched ranks
conda activate traffic
export SUMO_HOME=$HOME/sumo-1_20_0; export SLURM_MPI_TYPE=pmix
srun python -m mpi4py.futures -m src.ga.parallel  ...  # the parallel GA
```

The `-m mpi4py.futures` launcher starts an `MPIPoolExecutor`: **rank 0 is the master** that holds the GA state, and the remaining $w = (\text{ranks} - 1)$ ranks are **workers**. Every generation the master scatters the population's plans across the workers, each worker runs SUMO in-process on its plans and returns the delays, and the master gathers them — the master–worker pattern of the parallelisation section. Each job appends its measurements to a CSV on the node; the plots in this report are rendered *locally* from those CSVs (Matplotlib is deliberately not installed on the cluster, so a job's run is never blocked by plotting).

How the variables were swept.

- *Strong scaling* and *throughput* (Tables 2, 3): the city is fixed at 8×8 and the run is repeated for worker counts $w \in \{1, 3, 7, 15, 23, 31, 47\}$, so the same problem is solved with a growing pool.
- *Problem size* (Table 4): the worker count is fixed at $w = 23$ and the city is swept from 3×3 (9 intersections, $n = 18$) to 32×32 (1024 intersections, $n = 2048$).

The interactive dashboard demonstration in Sec. 10 instead uses a smaller 3×3 grid so the optimisation completes in seconds on a laptop; it is a usability demo, not an HPC measurement.

9 Results

9.1 Optimisation quality

On the 8×8 city the GA reduces total vehicle delay from 39 473 veh s (fixed-time) to 28 689 veh s, an improvement of **27.3** %. The same optimised plan is produced regardless of worker count (Table 2), confirming that parallelisation changes only the time-to-solution, not the solution itself.

9.2 Strong scaling

Here the problem is fixed (8×8 city, population 128, 20 generations) and the worker count is increased. Table 2 and Fig. 2 report the wall time, the resulting speedup $S(w) = t_1/t_w$ and the parallel efficiency $E(w) = S(w)/w$.

Table 2: Strong scaling on the 8×8 city (fixed problem, growing workers).

Workers w	Wall time [s]	Speedup S	Efficiency E
1	1291.69	1.00	1.000
3	437.04	2.96	0.985
7	199.51	6.47	0.925
15	100.61	12.84	0.856
23	72.01	17.94	0.780
31	60.09	21.50	0.694
47	47.92	26.96	0.574

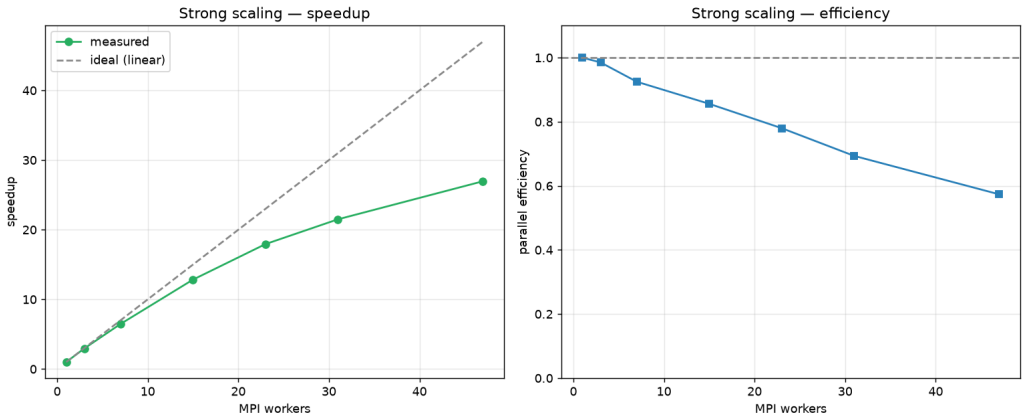

Figure 2: Strong scaling: speedup and parallel efficiency versus worker count. Speedup stays near-linear up to $\sim$15 workers and reaches 26.96$\times$ on 47 workers; efficiency declines as the fixed serial fraction of Eq. (2) grows in relative weight.

The behaviour is textbook Amdahl. Up to 15 workers the system is over 85 % efficient; beyond that, the per-generation serial overhead (scatter/gather plus the wait for the slowest worker in each batch) becomes a larger share of an ever-shorter runtime, so efficiency tapers to 57 % at 47 workers. The practically interesting point is the time-to-solution: a run that takes 21.5 min serially completes in 48 s on 47 workers.

9.3 Throughput

Throughput measures raw evaluation rate: how many SUMO runs per second the pool sustains as workers are added (fixed 8×8 city, 480 evaluations).

Table 3: Throughput on the 8×8 city (`libsumo` backend).

Workers w	Evaluations / s	ms / evaluation
1	1.447	691.1
3	4.339	230.5
7	9.981	100.2
15	20.359	49.1
23	28.940	34.6
31	36.103	27.7
47	45.776	21.8

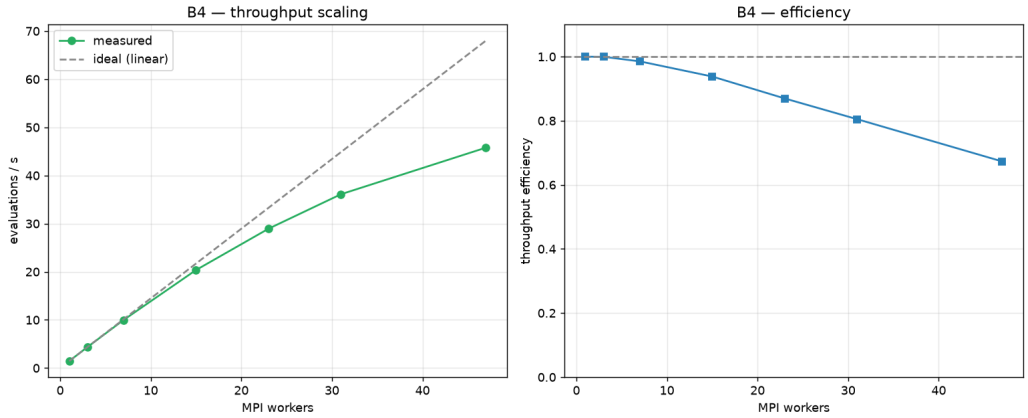

Figure 3: Throughput: sustained SUMO evaluations per second against worker count. The rate rises from 1.4 evaluations/s on a single worker to 45.8 evaluations/s on 47 workers — a 31× increase in raw evaluation capacity.

Throughput climbs from 1.4 evaluations/s (one worker, 691 ms per simulation) to 45.8 evaluations/s on 47 workers. The in-process `libsumo` backend is what makes each evaluation cheap enough for this to matter; a socket-based backend would spend a large fixed cost per run and cap the achievable rate.

9.4 Problem size

Finally we hold the worker count fixed (23) and grow the city, measuring how the cost per evaluation scales with the number of intersections.

Table 4: Cost per evaluation versus city size (23 workers, `libsumo`).

Grid	Intersections	Genome n	Evaluations / s	ms / eval
3	9	18	37.048	27.0
5	25	50	35.969	27.8
8	64	128	21.383	46.8
12	144	288	12.815	78.0
16	256	512	9.022	110.8
24	576	1152	6.090	164.2
32	1024	2048	4.730	211.4

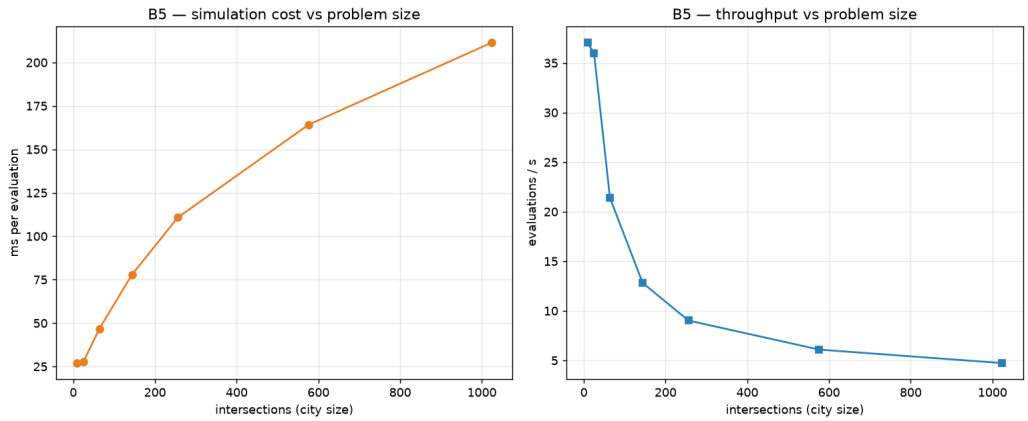

Figure 4: Problem size: evaluation rate and per-evaluation cost as the city grows from 9 to 1024 intersections. The cost per evaluation rises sub-quadratically with city size, so larger cities remain tractable on the same worker pool.

A 32×32 city has $113\times$ more intersections than a 3×3 grid, yet the cost per evaluation grows only about $8\times$ (from 27 ms to 211 ms). The simulator scales gracefully with the network, so optimising realistically large cities is a matter of adding workers, not of hitting an algorithmic wall.

10 The interactive dashboard

The dashboard is the project's centrepiece: it lets a user build a network, optimise its signals, and *see* the effect, without touching the command line. The screenshots below are from a 3×3 demonstration grid (baseline 15 638 veh s, optimised 12 124 veh s, $-22.5\,\%$).

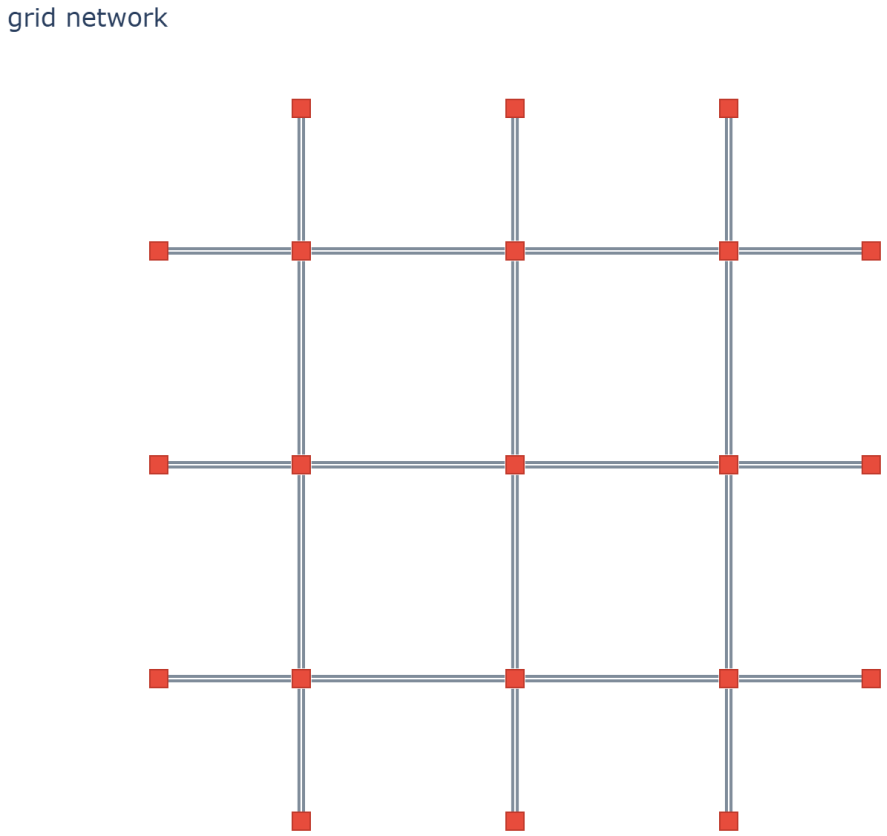

Figure 5: Network builder. The left panel parametrises the city — topology, junctions per side, lanes, edge length, demand intensity and the simulation horizon. Clicking *Build & show network* generates the SUMO network and renders it. Here a 3×3 grid with single-lane roads and a vehicle every 1.6 s is created.

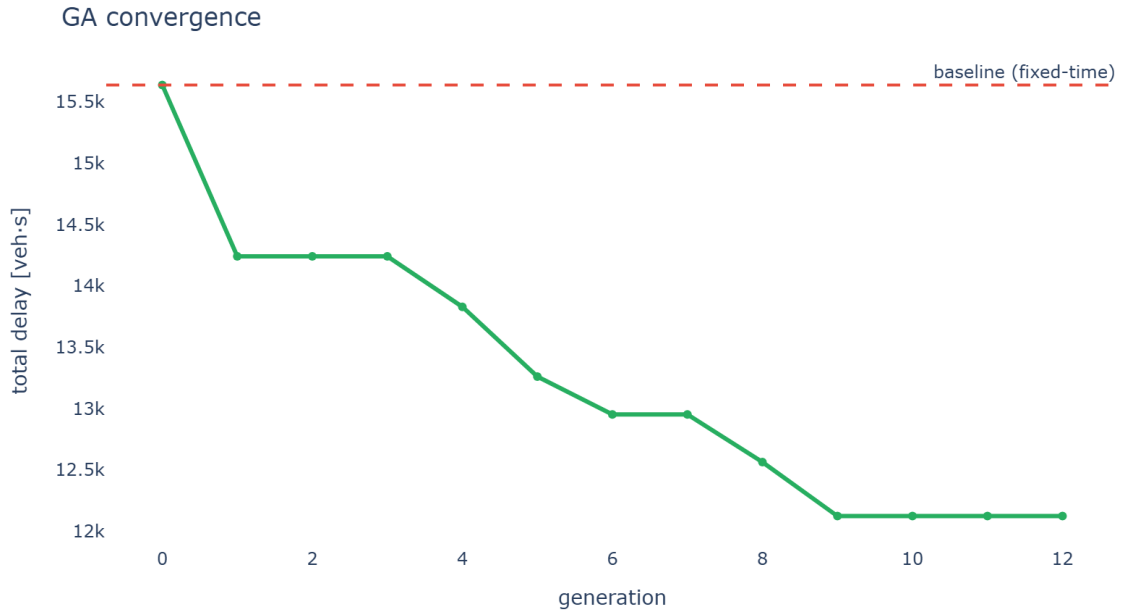

Figure 6: Optimisation result. The headline cards report baseline, optimised and improvement ($-22.5\,\%$). *Left:* the GA convergence curve — total delay drops generation by generation and plateaus, indicating the search has converged. *Right:* the same before/after delay as bars.

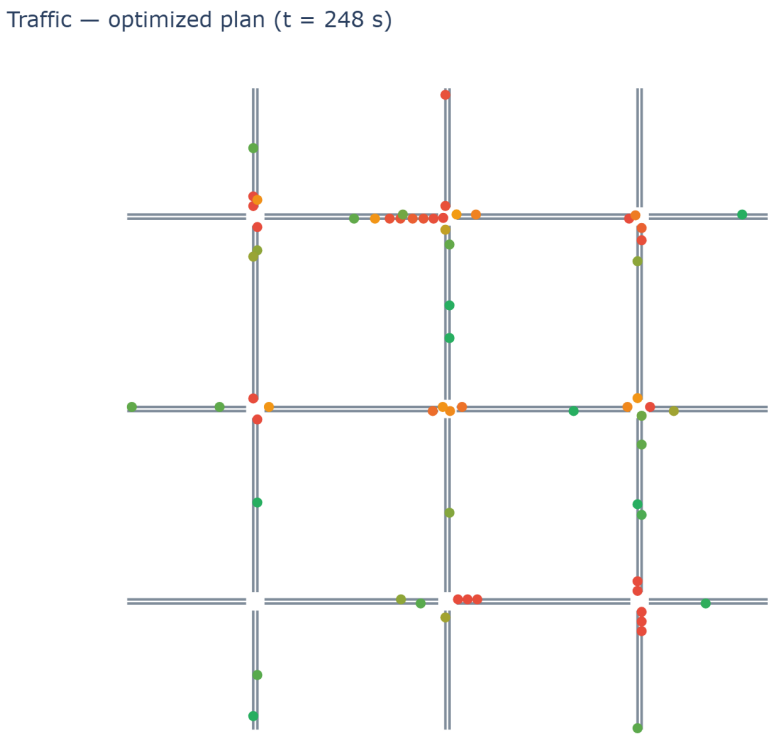

Figure 7: Traffic animation. The optimised plan is replayed: each dot is a vehicle, coloured by speed (green = moving, red = halting). A time slider scrubs through the simulated horizon, making queue formation and clearance visible directly.

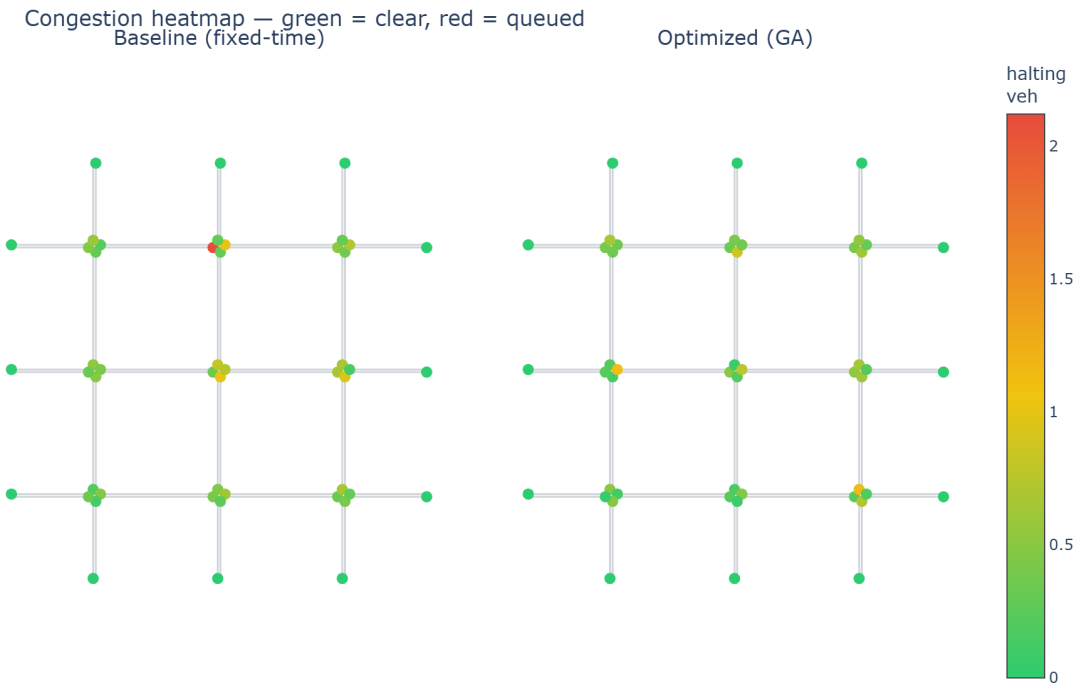

Figure 8: Congestion heatmap. Baseline (left) versus optimised (right), with each intersection coloured by its average number of halting vehicles (green = clear, red = queued). The optimised network shows visibly fewer hot spots — the same −22.5 % improvement, now localised in space.

The dashboard also supports a Cyfronet round-trip: a network can be exported as a bundle, trained on Ares with the parallel GA, and the resulting plan imported back to redraw the be-

fore/after charts — closing the loop between the single-machine front-end and the HPC back-end.

11 Discussion

Three threads come together in this project. First, framing signal timing as black-box minimisation of simulated delay (Eq. (1)) makes the problem precise and lets a generic optimiser attack any network the dashboard can build. Second, the algorithm comparison (Sec. 6) shows the GA is not an arbitrary choice: on an equal budget it beats random search, hill climbing and simulated annealing, reaching $-27.3\,\%$ where the others reach $-19.7\,\%$ or less. Third, the independence of fitness evaluations turns the optimiser into an HPC workload that scales to a $26.96\times$ speedup and 45.8 evaluations/s on Ares, while the problem-size study confirms the simulator itself scales gracefully to thousand-intersection cities.

The efficiency decline at high worker counts (Sec. 9.2) is the expected consequence of Amdahl's law: with evaluation so cheap under `libsumo`, the fixed per-generation serial cost eventually dominates. This is a feature of the regime, not a defect — the practically relevant metric, time-to-solution, still falls from over twenty minutes to under a minute.

12 Conclusions

We built a complete, reproducible pipeline — dashboard, genetic algorithm and SUMO simulator — that optimises urban traffic-signal timings and scales the optimisation on a supercomputer. The optimised plans cut total vehicle delay by $27.3\,\%$ on an 8×8 city and $22.5\,\%$ on the dashboard demonstration, all by retiming existing green phases without any new infrastructure. An equal-budget comparison against three alternative search strategies confirms that the genetic algorithm is the right tool for this noisy, gradient-free objective. By dispatching every fitness evaluation through an abstract parallel map, the very same optimiser runs unchanged on a laptop and on Cyfronet Ares, where it achieves a $26.96\times$ speedup on 47 workers and sustains up to 45.8 SUMOevaluations/s. Together these results show that high-performance computing makes data-driven signal optimisation practical at city scale: the bottleneck is no longer the search, but simply the number of workers one is willing to spend.